

Mara's Mad Monday

Mara I. Dumitru

Minerva University

CS110: Project 1

Rohan Shekhar

Nov 12, 2024

INTRODUCTION	3
TASKS	4
Regular Tasks	4
Fixed Tasks	4
ALGORITHMIC STRATEGY	5
Personalized Modifications to Scheduler	5
Video	5
LBA TASK DOCUMENTATION	6
ALGORITHMIC ANALYSIS	11
Prior to Test Drive	11
Advantages	11
Failures	12
Scheduler Output Efficiency & Comparisons	12
Scheduler Algorithm Efficiency & Comparisons	14
SCHEDULER IMPROVEMENTS	17
List of potential Improvements:	17
Paragraph of Concrete Improvement in Algorithm:	17
REFERENCES	19
AI STATEMENT	20
APPENDIX	22
Part I: Reflection	22
Part II: HC/LO Applications	22
Part III: Python Code	23

INTRODUCTION

This paper is a descriptive analysis of a Python implementation of a task scheduler. To make this assignment more efficient in terms of the Utility of the output for my personal use, I added a couple of extra special mods (additional improvements to the original task scheduler) specifically geared toward my interests in a task scheduler. Such modifications include having a fixed time I finish tasks by so that I get to sleep, or making sure to go to a Night market at night and not at 7am. While this assignment posed several challenges during its lengthy process, I conclude to be content with the result of this paper, and I hope you enjoy!

TASKS

Regular Tasks

ID	Title	Duration (in hours)	Individual	Category	Dependent	Location	Right time constraint	Left time constraint
1	Get Dressed	1	yes	Chores	None	no	None	None
2	NTU Library Study Session	1	no	Non-Minerva Academics	1, 4, 9	no	None	None
3	NTU Piano	0.5	no	Recreation	1, 4	no	None	None
4	Bike to NTU	0.5	yes	Chores	1	yes	None	None
5	Read	0.75	yes	Recreation	1	no	None	None
6	Lunch	1.5	yes	Food	1, 10, 12	yes	15	None
7	AI Lab Study	1	no	Non-Minerva Academics	1, 9	no	None	None
8	Night Market	2	no	Recreation	1, 4, 10	yes	None	16
9	Charge Electronics	0.75	yes	Chores	1	no	None	None
10	Extract Cash	0.1	yes	Chores	1	no	None	None
11	Breakfast	1	yes	Food	1, 10, 12	no	10	None
12	Brush Teeth	0.05	yes	Chores	None	no	None	None
13	CS110 Pre-Class Work	3	yes	Minerva Academics	9	no	None	None
14	CS111 Pre-Class Work	1	yes	Minerva Academics	9	no	None	None
15	SS111 Pre-Class Work	0.5	yes	Minerva Academics	9	no	None	None
16	Taiwanese Pickle Sandwich	0.25	yes	Food	1, 4, 8, 10	yes	None	None
17	Watch a Movie	2	yes	Recreation	None	no	None	None

Fixed Tasks

ID	Title	Duration (in hours)	Individual	Category	Dependent	Location	Time constraint
18	AI Lab Meeting	1	no	Non-Minerva Academics	7	no	18

Location Based Task Engagement:

Biking to NTU: Biking through the city as I reach my final destination engages me with the city by exposing me to street life culture in Taiwan, as without biking for example I would not notice how Taiwan is motorbike dominated on streets.

Night Market: Night markets provide a way to experience night life in Taiwan, they not only provide a window to try out all kinds of taiwan food, but observe different products like clothing or accessories, as well as see people just having fun like dancing as a community

Taiwanese Pickle Sandwich: The Taiwanese Pickle sandwich is one of a kind it engages me with the city by trying out local cuisine

Lunch: Similar as the sandwich Lunch engages me with the city as usually I just find lunch close to wherever I am, whatever inspires me. So it engages me with the given location by seeing how food differs across taiwan. Like around NTU for instance there is cheap and western food to accommodate young broke college students, and around financial shopping district there are very expensive restaurants to accommodate consumers.

ALGORITHMIC STRATEGY

Personalized Modifications to Scheduler

Additional attributes:

[individual, category, time-constraint-left, time-constraint right, time-stamp]

Reasons for additional attributes mentioned in Appendix Part III Task Classes code, or in the video description

Other additions:

Goodnight: time I finish tasks by, if time hits this goodnight time, then stop completing tasks

Goodnight - 2: Make sure to finish academics 2 hours before sleeping

Wake-up: time I wake up

Username: The user's name

Day of the week: The day of the week

Video

<https://youtu.be/7EE6SXQr88o>

LBA TASK DOCUMENTATION

ID	Algorithm Time Stamp	Actual Time Stamp	Photo	Caption
1	10:00 AM	10:00 AM		Getting Dressed on time, great start to my day!
10	11:00 AM	11:02 AM		Extracting cash at my local 7 eleven and drinking yakurt
9	11:06 AM	11:08 AM		Charging my phone for the day

4	11:51 AM	11:51 AM		About to start biking to NTU!
12	12:21 PM	None	Not Executed	I meant to bring my tooth brush to NTU however I forgot leading to a minor complication and so I could not complete this task
15	12:24 PM	12:26 PM	None	Forgot to take photo
2	12:54 PM	12:54 PM		Studying at the library with friends
7	1:54 PM	1:54 PM		Me struggling with a VAE for my AI Lab at a cafe

3	2:54 PM	2:57 PM		Playing a piano and violin duet!
14	3:54 PM	3:54 PM		Working on my CS111 preclass work, just to skip the class the next day to finish this assignment
8	4:24 PM	4:33 PM		Hanging out with my friend at a night market and exploring different food and cloth stands together.

18	6:00 PM	6:00 PM		Taking my meeting at a night market as I had to halt my night market task in the middle!
16	7:24 PM	7: 24 PM		Finally eating my cold Taiwanese Pickle Sandwich. I had to wait like 10 minutes to eat it, just to show the time stamp haha!
5	7:39 PM	7:40 PM		Too tired to read, but read anyways!

17	8:24 PM	8:31 PM		Started watching a movie late because I forgot to include showering in my task schedule
6	None	None	Not Part Of Schedule	Skipped task because not enough time in the day
11	None	None	Not Part Of Schedule	Skipped task because not enough time in the day
13	None	None	Not Part Of Schedule	Skipped task because not enough time in the day

ALGORITHMIC ANALYSIS

Prior to Test Drive

See test cases in the code in appendix Part III (headed text above the actual code), to see how demonstrated code schedules tasks based on priority, and how changing order does not matter.

Note I just noticed that you required an assert statement however, at the time of coding I missed this information. I apologize. However, to create an assert statement it would be pretty easy, as I would store the output as a string and compare the two strings with an assert statement.

I think my schedule looks reasonable. Please note that it took several iterations of changing code and adding more things to get the output to look sensible, as originally I would eat breakfast at 6 pm, and go to the night market at noon, which was not reasonable for me. I added modifications like left and right time constraints to mitigate this. Otherwise, I like how my studying is spaced out, with intermediary tasks like playing piano in between, as I do this usually as well.

Furthermore, my chores are more towards the beginning of the day which is really nice too! The only issue in the output is that I brush my teeth at NT (after biking there),, which is still feasible as I can bring my toothbrush to NTU. However this is not preferable, and an improved version of a scheduler would mitigate this.

Advantages

An advantage specifically for me is that I am pretty bad at planning, I usually "go with the flow," which results in me not doing everything I should have done in a given day. By creating this I could see the plan for my day and follow it so I could get what was prioritized to get done in the day. The advantage of this is that since it is an algorithm it works perfectly every time, instead of

a human (like me) who can make mistakes when planning. By following this output I think I got more done this Monday than my usual Monday (but that might be because in general, I do not really plan).

Failures

I was supposed to brush my teeth at NTU according to my algorithm. Two failures resulted from this. First was a failure in the algorithm, right before I brushed my teeth I biked to NTU I would have preferred to brush my teeth at the res hall than bike to NTU, so in the algorithm this means that the utility value assigned to each task could be improved. The second failure was on my side, I forgot to get my toothbrush from the residence hall, which resulted in my inability to complete this task. Thankfully this was a short task so it did not affect the timeline, I just waited around and talked to my friend for a few minutes.

Scheduler Output Efficiency & Comparisons

Here is a schedule designed by ChatGPT, you can view my prompts in the AI Statement section:

10:00 am - 10:05 am: Brush Teeth (Task 12, Chores)

10:05 am - 10:15 am: Extract Cash (Task 10, Chores)

10:05 am - 10:35 am: Get Dressed (Task 1, Chores)

10:35 am - 11:20 am: Breakfast (Task 11, Food)

11:20 am - 11:30 am: Extract Cash (Task 10, Chores)

11:30 am - 12:30 pm: NTU Library Study Session (Task 2, Non-Minerva Academics)

12:30 pm - 1:00 pm: Bike to NTU (Task 4, Chores)

1:00 pm - 1:45 pm: Lunch (Task 6, Food)

1:45 pm - 2:45 pm: AI Lab Study (Task 7, Non-Minerva Academics)

2:45 pm - 5:45 pm: CS110 Pre-Class Work (Task 13, Minerva Academics)

5:45 pm - 6:00 pm: Charge Electronics (Task 9, Chores)

6:00 pm - 7:00 pm: AI Lab Meeting (Task 18, Fixed Task)

7:00 pm - 9:00 pm: Night Market (Task 8, Recreation)

9:00 pm - 9:45 pm: Read (Task 5, Recreation)

9:45 pm - 10:00 pm: Taiwanese Pickle Sandwich (Task 16, Food)

10:00 pm - 12:00 am: Watch a Movie (Task 17, Recreation)

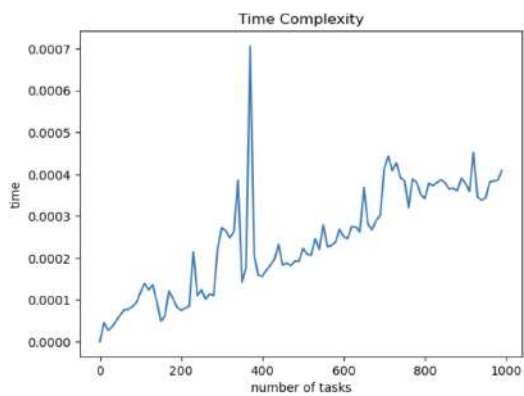
However, ChatGPT changed some of the durations of the tasks, and unfortunately, I do not have the latest version to fix this. I can compare this to the schedule produced by my algorithm, as ChatGPT's schedule follows a more efficient way to place my tasks, such as brushing my teeth before biking to NTU, as it mitigates forgetting my toothbrush at the residence hall, and follows a more human way to schedule my tasks. Furthermore, even if it cheated by making my tasks shorter it has a 100% completion rate and no downtime.

That being said I would say that the output of my algorithm is very efficient as I got most of my tasks done, as I completed 15 out of 18 tasks in my day, leading to an 83.3% completion rate of tasks. Furthermore, the task category that I consider the highest priority of academics also had a high percentage completion rate of 83.3% (like my preclass work, studying, and AI lab), showing how I maximize the utility of my tasks. The one that I skipped is the longest so I made sure that I could both maximize the number of academic tasks and maximize the amount of time to do other tasks too. Additionally, another metric that shows the efficiency of my algorithm is

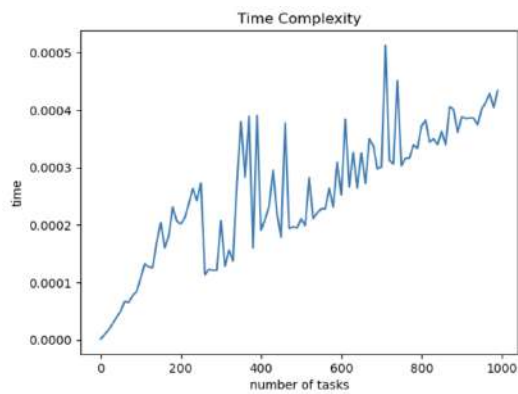
that I did not leave gaps in time. Especially when it came to scheduling fixed tasks, my output has me doing tasks until the required time of my fixed tasks, and then resuming the regular task. The combination of having to waste time and getting to my fixed task on time makes my output efficient.

Scheduler Algorithm Efficiency & Comparisons

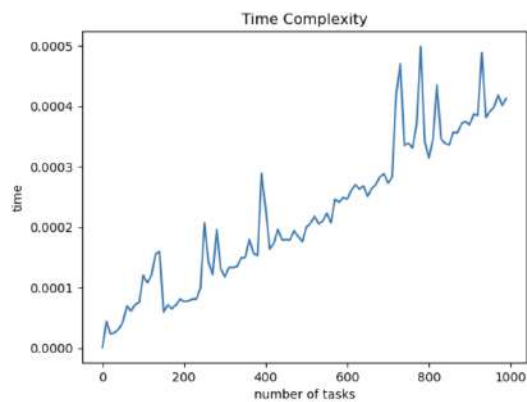
Experiments:



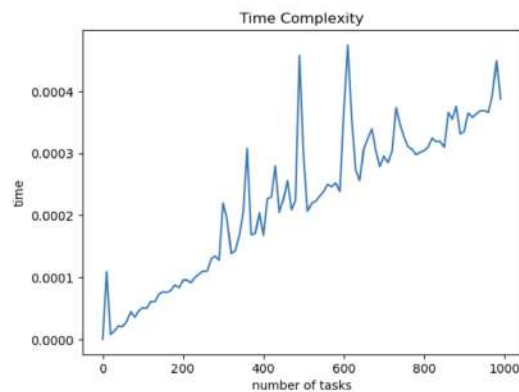
A.



B.



C.



D.

I designed 4 empirical experiments where I assign different utility values to tasks (using duration) and increase the number of tasks. Figure A has no dependencies assigned for each task, B has dependencies in increasing order (i.e. task 3 is dependent on task 2, task 2 is dependent on task 1), C has dependencies in random order, and D has dependencies in decreasing order.

They all follow the same time complexity, showing how the dependencies do not affect the run time of the algorithm, which is because the time complexity of calculating dependencies is equal to the time complexity of the rest of the algorithm as tasks with dependencies are pushed into the heap with $O(\log n)$ time complexity, just like any other task. Then since this is done over max of n tasks the total time complexity would be $O(n \log n)$. However, when looking at the empirical data we can see that they appear to follow a more linear trend of time complexity of $O(n)$. Let me explain below.

To push a task into a heap it takes $O(\log n)$ time complexity in the worst case scenario, as when we push a task we assign it to the rightmost leaf, in the worst case scenario for a min-heap this task would have the highest utility value, and so we would want it at the root of our max heap. In this case, we would have to sift this leaf through the entire length of the tree making the same number of comparisons as the height. Since a heap, is a nearly complete binary tree, it has height $\log n$, therefore the complexity of this is $O(\log n)$, which is an upper bound for complexity. In the best-case scenario we would only perform one comparison and see that our task is smaller than its parent and keep it there leading to $\Omega(1)$ efficiency, which is a lower bound for complexity. However, on average, we would sift through some of the height of the tree leading to an average time complexity of $\Theta(\log n)$, which is a tight bound for complexity. To perform heap push we just

push out the root of the tree, which in both best and worst cases has a time complexity of $O(1)$, meaning that it has a time complexity of $\Theta(1)$. To heapify the time complexity is also $O(\log n)$, for similar logic as heap push, so I will not go into depth about this.

The overall time complexity to build a heap is $O(n)$, although logically since there are n different tasks and each operation takes a max of $O(\log n)$ efficiency, this would take time complexity of $O(n \log n)$. However even though each operation has an efficiency of $O(\log n)$ and there are n elements, typically, the complexity of each operation for each task decreases as you get closer to the root, as the height decreases as well.

SCHEDULER IMPROVEMENTS

List of potential Improvements:

1. improving group project's utility calculation to factor group size
2. creating a max heap for left time constraint tasks instead of a list
3. creating a max heap for tasks with dependencies instead of a list
4. account for several fixed tasks at a given time
5. handling tasks with the same priority
6. include multitasking as another attribute
7. updating priority values if skipped a task with same category (like if I skipped breakfast priority for lunch should be higher)
8. directly adding left time constraint tasks to the heap with left tasks
9. directly adding tasks with dependencies to the heap with dependency tasks

Paragraph of Concrete Improvement in Algorithm:

I will choose to focus on Improvements #2 and #8 in more depth. At the given state if a task is required to be completed after a given time, however, its priority value tells the algorithm to execute it sooner, it appends the task to the list `self.left_time_tasks`. Through each iteration of the heap pop method the algorithm if any of the tasks in `self.left_time_tasks` have their left time constraint met, and if so it removes the task from the list and pushes it back into the heap. However, this is deemed to be inefficient as the algorithm checks through all the tasks in the `self.left_time_tasks`, however, has an efficiency of $O(L)$ where L is the number of left time constraint tasks. The scale of a daily scheduler does not matter in terms of efficiency as there are few tasks, however, if I would want to provide a weekly schedule with more tasks this would

scale linearly. A more efficient way to do this would be to create a new min heap (which would have a similar code structure as the fixed tasks heap) with these tasks and pop out the task with the smallest left time, this way we do not need to check all the tasks, leading to an efficiency of $O(1)$. Furthermore, to further improve efficiency, instead of popping left constraint tasks from the main task scheduler max heap and pushing them into the respective min heap, we can directly add them to the min heap. This would improve space complexity as we would only alter the main task heap with these time-constraint tasks only once, however, in the current condition we add the tasks to the main heap, then remove them, and then add them again. Which is less efficient. (Words: 299)

REFERENCES

Schulz-Hardt, S., & Brodbeck, F. (n.d.). Group Performance and Leadership.

<https://www.blackwellpublishing.com/content/hewstonesocialpsychology/chapters/chapter13.pdf>

Dube, S. (2020, October 1). No pain, no gain? Effect of short term pleasure on long term happiness. The Courier Online.

<https://www.thecourieronline.co.uk/no-pain-no-gain-effect-of-short-term-pleasure-on-long-term-happiness/>

Waseem. (2024, January 13). How to maintain a balance between academics and extracurricular activities | Growth Academy. Growth Academy.

<https://growthacademy.education/how-to-maintain-a-balance-between-academics-and-extracurricular-activities/>

Center, W. S. (2023, November 14). A Student's Guide to Getting Better Sleep (Even During Finals) | Whitney Sleep Center. Whitney Sleep Center.

<https://whitneysleepcenter.com/blog/a-students-guide-to-getting-better-sleep-even-during-finals/>

Shekhar,& Dumitru, Mara (2024, Oct 24), CS110 Session 13 - [7.2] Heaps and Priority Queues, Preclass work-book <https://forum.minerva.edu/app/courses/3282/sections/11989>

AI STATEMENT

1. I used ChatGPT to generate a schedule for me to evaluate the efficiency of the output of my scheduler. This is a valuable usage of AI as it shows a novel way AI can be used to enhance my work and help me criticize my work instead of impeding my learning.

Unfortunately, as I used an image of my task list in my conversation, and ChatGPT is unable to share conversations with file uploads, I am unable to provide a link to the conversation. However, I have included screenshots of my conversation on the next page.
2. I used Grammarly to edit my sentence level grammar
3. Otherwise I did not use AI to write this paper

Region: Home									
Task ID	Task Name	Category	Duration (h:m)	Priority	Dependencies	Start Time	End Time	Status	Notes
1	Brush Teeth	Chores	0:05	Low		10:00	10:05	Planned	
2	Extract Cash	Chores	0:10	Low		10:05	10:15	Planned	
3	Get Dressed	Chores	0:10	Low		10:20	10:30	Planned	
4	Breakfast	Food	0:10	Low		10:30	10:40	Planned	
5	NTU Library Study Session	Academics	1:00	High		11:30	12:30	Planned	
6	NTU Piano	Recreation	0:30	Medium		12:30	1:00	Planned	
7	Lunch	Food	0:15	Low		1:00	1:15	Planned	
8	AI Lab Study	Academics	0:30	Medium		2:00	3:00	Planned	
9	CS110 Pre-Class Work	Academics	0:45	Medium		3:00	6:00	Planned	
10	AI Lab Meeting	Fixed Task	0:10	High		6:00	7:00	Planned	
11	Night Market	Recreation	0:30	Low		7:00	9:00	Planned	
12	Read	Recreation	0:15	Low		9:00	9:45	Planned	
13	Charge Electronics	Chores	0:15	Low		9:45	10:00	Planned	
14	Watch a Movie	Recreation	0:20	Low		10:00	12:00	Planned	
15	Get Dressed	Chores	0:10	Low		10:00	10:10	Planned	
16	Extract Cash	Chores	0:10	Low		10:05	10:15	Planned	
17	Brush Teeth	Chores	0:05	Low		10:00	10:05	Planned	
18	Get Dressed	Chores	0:10	Low		10:20	10:30	Planned	
19	Breakfast	Food	0:10	Low		10:30	10:40	Planned	
20	NTU Library Study Session	Academics	1:00	High		11:30	12:30	Planned	
21	NTU Piano	Recreation	0:30	Medium		12:30	1:00	Planned	
22	Lunch	Food	0:15	Low		1:00	1:15	Planned	
23	AI Lab Study	Academics	0:30	Medium		2:00	3:00	Planned	
24	CS110 Pre-Class Work	Academics	0:45	Medium		3:00	6:00	Planned	
25	AI Lab Meeting	Fixed Task	0:10	High		6:00	7:00	Planned	
26	Night Market	Recreation	0:30	Low		7:00	9:00	Planned	
27	Read	Recreation	0:15	Low		9:00	9:45	Planned	
28	Charge Electronics	Chores	0:15	Low		9:45	10:00	Planned	
29	Watch a Movie	Recreation	0:20	Low		10:00	12:00	Planned	
30	Get Dressed	Chores	0:10	Low		10:00	10:10	Planned	
31	Extract Cash	Chores	0:10	Low		10:05	10:15	Planned	
32	Brush Teeth	Chores	0:05	Low		10:00	10:05	Planned	

These are a list of tasks that I want to do today starting from 10 am and ending at midnight. Note that all tasks cannot be done within this time frame so you have to schedule my tasks in order to maximize my productivity based on the different criteria of each task like category and individuality. Note that the duration of each task is listed in hours. Please create me a schedule to follow for the day and include the time I start each task. Remember I wake up at 10 am and go to sleep at midnight.

To create an optimal schedule for your day, I'll prioritize your tasks based on dependencies, category importance, and time constraints. Here's a step-by-step breakdown:

- Start Time and End Time:** You're awake from 10 am until midnight, giving a total of 14 hours (10 am - 12 am).
- Dependencies:** Tasks must follow any dependency chains specified.
- Fixed Tasks:** The "AI Lab Meeting" at 6 pm is a fixed task with a dependency on "AI Lab Study."
- Category and Time Constraints:** High-priority categories and tasks with specific right or left time constraints are scheduled accordingly.

I'll use this information to maximize your productivity within the available time. Let's proceed with scheduling based on the criteria:

Schedule:

- 10:00 am - 10:05 am: Brush Teeth (Task 12, Chores)
- 10:05 am - 10:20 am: Extract Cash (Task 10, Chores; minimal duration)
- 10:20 am - 10:30 am: Get Dressed (Task 1, Chores)
- 10:30 am - 11:30 am: Breakfast (Task 11, Food; constrained to start before 10:00 am)
- 11:30 am - 12:30 pm: NTU Library Study Session (Task 2, Non-Minerva Academics; high academic priority)
- 12:30 pm - 1:00 pm: NTU Piano (Task 3, Recreation; break after academic session)
- 1:00 pm - 2:00 pm: Lunch (Task 6, Food; specified to be after Task 1, Task 10, and Task 12)
- 2:00 pm - 3:00 pm: AI Lab Study (Task 7, Non-Minerva Academics; needed before the fixed AI Lab Meeting)
- 3:00 pm - 6:00 pm: CS110 Pre-Class Work (Task 13, Minerva Academics; long duration and academic priority)
- 6:00 pm - 7:00 pm: AI Lab Meeting (Task 18, Fixed Task; scheduled for this exact time)
- 7:00 pm - 9:00 pm: Night Market (Task 8, Recreation; left time constraint set for 4 pm)
- 9:00 pm - 9:45 pm: Read (Task 5, Recreation; relaxation time)
- 9:45 pm - 10:00 pm: Charge Electronics (Task 9, Chores; to prepare for the next day)
- 10:00 pm - 12:00 am: Watch a Movie (Task 17, Recreation; ending before midnight)

This schedule accommodates all constraints and dependencies, maximizes productivity by balancing academic and recreational tasks, and completes as many tasks as possible within your available time. If you'd like, I can add further details on how tasks were prioritized.

Actually make sure you keep track of dependencies like the dependencies column are the task ids that need to be done before that task can be executed

Thank you for pointing that out! I'll go through each task carefully and ensure that dependencies are respected. Let's build the schedule while adhering to the dependency requirements, fixed time constraints, and optimizing based on productivity.

Revised Schedule (with Dependencies Respected)

- 10:00 am - 10:05 am: Brush Teeth (Task 12, Chores)
No dependencies.
- 10:05 am - 10:15 am: Extract Cash (Task 10, Chores)
Dependent on Task 1 (Get Dressed), which hasn't been done yet, so we'll skip this for now.
- 10:05 am - 10:35 am: Get Dressed (Task 1, Chores)
No dependencies, required for many tasks.
- 10:35 am - 11:20 am: Breakfast (Task 11, Food)
Dependent on Tasks 1, 10, and 12 (Get Dressed, Extract Cash, and Brush Teeth). With Brush Teeth and Get Dressed done, I'll fit in Extract Cash next.
- 11:20 am - 11:30 am: Extract Cash (Task 10, Chores)
Dependent on Task 1 (Get Dressed), which is now done.
- 11:30 am - 12:30 pm: NTU Library Study Session (Task 2, Non-Minerva Academics)
Dependent on Tasks 1, 4, and 9 (Get Dressed, Bike to NTU, Charge Electronics). Bike to NTU will be prioritized next.
- 12:30 pm - 1:00 pm: Bike to NTU (Task 4, Chores)
Dependent on Task 1 (Get Dressed), which is done.
- 1:00 pm - 1:45 pm: Lunch (Task 6, Food)
Dependent on Tasks 1, 10, and 12, which are all done.
- 1:45 pm - 2:45 pm: AI Lab Study (Task 7, Non-Minerva Academics)
Dependent on Tasks 1 and 9 (Get Dressed and Charge Electronics).
- 2:45 pm - 5:45 pm: CS110 Pre-Class Work (Task 13, Minerva Academics)
Dependent on Task 9 (Charge Electronics) — I'll fit this task in next.
- 5:45 pm - 6:00 pm: Charge Electronics (Task 9, Chores)
Dependency for several tasks.
- 6:00 pm - 7:00 pm: AI Lab Meeting (Task 18, Fixed Task)
Dependent on Task 7 (AI Lab Study), which is complete.
- 7:00 pm - 9:00 pm: Night Market (Task 8, Recreation)
Left time constraint of 4 pm.
- 9:00 pm - 9:45 pm: Read (Task 5, Recreation)
Dependent on Task 1 (Get Dressed), which is complete.
- 9:45 pm - 10:00 pm: Taiwanese Pickle Sandwich (Task 16, Food)
Dependent on Tasks 1, 4, 8, and 10, all of which are done.
- 10:00 pm - 12:00 am: Watch a Movie (Task 17, Recreation)
No dependencies, fitting nicely into the end of the day.

Summary of Changes

- I adjusted the order to ensure all dependencies are met.
- Ensured fixed time for "AI Lab Meeting" at 6 pm.
- Accounted for each dependency chain before scheduling dependent tasks.

This revised schedule respects each task's dependency and keeps productivity high by balancing between academic, chores, and recreational tasks.

APPENDIX

Part I: Reflection

From Session 13, something that helped me clarify my problem-solving approach was when we discussed the time complexity of extracting the maximum element from a max-heap, which is $O(1)$ whereas extracting the maximum element from a list would be $O(n)$ as we would have to scan through the entire list to find the maximum, and sorting the list has minimum $O(n \log n)$ complexity. This helped me identify using a max-heap was necessary.

Part II: HC/LO Applications

#professionalism: My grammar is correct, I included an AI Statement, my formatting is professional, I even included headers in my python code to separate sections, I used a proper tone, I followed the guidelines of the assignment description, I cited work, and I stayed within word/video length count when specified. (Words: 50)

#cs110-AlgoStratDataStruct: In my Algorithmic-Strategy Video, I explain how my algorithm works without using jargon. I explain using simple concepts [0:26] including visualizations to help the consumer understand. I show my comprehension of how my code works. I explain why the chosen solution is better than an alternative [0:10]. (Words: 49)

#cs110-CodeReadability: I included the doc strings and comments. I was consistent in using descriptive and conventional names. I exceeded in this HC application as my output is formatted very neatly (descriptive, colored, bolded, titled), I made sure all the logic of my code is visible in this PDF and included Headers. (Words: 50)

#cs110-ComplexityAnalysis: I differentiate between average, worst, and best-time case scenarios and use asymptotic notation to describe them. I explain how to avoid the worst-case

scenario, and when it occurs. I provide a step-by-step explanation of how I found the time complexity both in the video and in the scheduler efficiency description. (Words: 50)

#cs110-ComputationalCritique: I explain why I chose to use priority queues and heaps to build my scheduler in my video. Furthermore, in the scheduler improvements section, I describe how an improvement would be more efficient in terms of time complexity and space complexity and contrast it with the current solution. (Words: 48)

#cs110-PythonProgramming: I write a novel code to solve the given task. I went above and beyond I believe because I added other factors to my code not described in the problem (i.e. time constraint (bedtime) to complete tasks by, `left_time_constraint`). I provide test cases to test my algorithm. (Words: 48)

#communicationdesign: I stick to a simple soft color ballet whose different elements highlight the principle of informative change. Following the principle of limited capacity, I keep to 1 diagram per slide (instead of multiple), and change slides frequently to keep the consumer engaged in my description. I chose visual slides with little text. This is a particularly strong application as without these visual slides my verbal contribution would be less strong, which compliments the LOs applied in this assignment. (Words: 78)

Part III

1 Formatting

Quick Note on formatting (#cs110-CodeReadability). I chose to only format the actual logic of the code so that it is all visible, as in no lines are cut off in the PDF. I did not chose to continue this with comments and print statements as this would create too much of a clutter and the code would be too long (in height) to follow along. If curious about these aspects you can download and see the ipynb file. Arguably this attention to detail in addition to the nicely formatted print statement goes outside class materials and is an above and beyond application of #cs110-CodeReadability.

```
In [4]: """
importing line formating libraries (so that the number of characters in a
"""
%load_ext blackcellmagic
```

```
In [5]: """
importing text formating libraries, if code does not work make sure to !p
"""

from rich import print
from rich.console import Console

console = Console()
```

```
In [6]: def OClock(number):
        """
        Takes a number and returns the number as time in OClock notation like

        Parameters
        -----
        number: float
            time in hours

        Returns
        -----
        string
            time in 12-hour oclock notation
        """
        if (
            number >= 13 and number < 24
        ): # case for when number is greater than 13 to represent in 12-hour
            time_oclock = number - 12
            ampm = " PM"
        elif (
            number >= 24
        ): # case for when number exceeds 24 in the case I go to sleep at 1a
            time_oclock = number - 24
            ampm = " AM"
        else: # AM time
            time_oclock = number
```

```

    if number >= 12:
        ampm = " PM"
    else:
        ampm = " AM"

    hour = int(time_oclock) # represents hour as the int value of the fl
    minute = round(
        (time_oclock - int(time_oclock)) * 60
    ) # minute as the decimal part of the float multiplied by 60

    if minute < 10: # formatting for the minute to look better
        time_oclock = f"{hour}:0{minute}{ampm}"
    else:
        time_oclock = f"{hour}:{minute}{ampm}"

    return time_oclock # returns time in 12-hour notation

```

2. Regular and Fixed Task Classes with Utility Values

How to calculate Utility

Fields Not Used but Defined: status

While this field is defined, I never actually use it in my class as part of the logic in my code as I did not deem it necessary.

Extra Data Fields: individual, category, time constraint left, time constraint right, time stamp

For regular tasks I introduced 2 new data fields to help with my priority calculations "individual" and "category," I have also incorporated duration and location based into my priority calculations for a given task.

A task can be in any of the categories: ("Chores", "Non Minerva Academics", "Minerva Academics", "Recreation", "Food"). Recreation has the least utility as it provides short term happiness, and is not productive towards long term happiness which has more utility for a given person (Dube, 2020). Next Chores is on the list, I can skip chores and just do them the next day if I have higher utility tasks. Next is "Food" I prioritise food over Chores, as it is an essential to maintain life, but personally realistically I prioritize my academics over eating so I wanted my model to reflect this personal choice, it has not data to reflect this. To maintain focus on long term goals and and to maximize academic future it is recommended to prioritize university academics over other academics, which is why tasks with category Minerva Academics have a higher utility value added to them than Non Minerva Academics (Waseem, 2024).

A task can either have individual property set as "Yes" or "No," if it is set as "Yes" it is considered a task that I do individually, and if set as "No" it is a task I must do with another person. Schulz-Hardt & Brodbeck describe in their detailed report how group projects have a higher outcome than individual projects through the successful coordination of the group members. Coordination loss in group projects and why

group projects tend to fail is when individual members do not put their individual contributions, which is why group projects should be prioritized over individual projects with respect to the number of members in the group (Schulz-Hardt & Brodbeck, n.d.). My given tasks that require a group effort operate in trios which is why I have assigned 3 times the utility value of individual tasks (see Limitations section for further improvements).

A task can be location based or not, this matters in the utility value of task as if two tasks have the same utility then the location based task should be prioritized, as one may not be able complete this task in another given time frame. However, this property is not a big deciding factor for a Tasks utility value, so the added utility is very small.

Let us say given tasks a and b have utility x and duration of 1 hour, and task c has the same utility x however its duration is 2 hours. Reasonably we would want my algorithm to prioritize tasks a and b as combined they would have a utility of $2x$ for the same duration of task c, so we would want to incorporate duration into the priority calculation of each task, reasonably we would divide the utility of a task by its duration to update its utility so that it is inversely proportional to its duration. However, we do not want the duration to undermine a very important tasks utility so to mitigate this instead we divide by the square root of the duration.

Additionally for regular tasks I introduced two new fields "time_constraint_right" and "time_constraint_left." Time constraint right is the time a task must be completed by, for example I do not want to have Breakfast after 10 AM (otherwise I consider it Lunch), however I wanted my algorithm not to do these tasks unless they had a higher priority value than other tasks, so if I have a lot of academics I will probably skip Breakfast (which is realistic), and have a bigger lunch. So if the time_constraint_right is not met, then I skip the task.

Time constraint left is the time a task must be completed after, for example it does not make sense to go to night market at 7 am in the morning (its a morning market) so by adding a left time constraint if the given task is highest in priority it will wait until the left time constraint is met until executing the task (see Limitations section for further improvements).

For fixed tasks they have a new field of "time_stamp," which is the required time that a task needs to happen. It first checks if the dependencies are met then executes the fixed task at the time stamp. If dependencies are not met then I skip this task, so if I have no completed CS110 preclass work I will not attend CS110 class, but I will not prioritize CS110 preclass work over other tasks just because a fixed task depends on it.

The paper instructions require me to base the utility of my tasks based on which tasks are dependent on these tasks which I have done by adding the corresponding utility values to the task that is a dependency. However, my code works perfectly well without this, and for my personal preference of a scheduler in my daily life I would probably not incorporate this into my tasks as showed why in the test cases.

The biggest personalization added to the task scheduler was adding a bed time

For my personal scheduler I have also incorporated a time constraint for when I go to sleep. Usually I have more tasks in a day than I can complete, so I wanted my scheduler to detail which tasks I should do today and in what order to maximize my utility. Additionally I cannot sleep if I work on Academics less than 2 hours in advance of going to bed, which is a common trend in a lot of college students (Center, 2023). So I wanted to add that additional constraint. The user can directly input this into the model, as well as their desired wake up time.

```
In [14]: class Task:
    """
    A class that stores properties of a daily task, and implements a method to
    create a task.

    Properties
    -----
    id : int
        the id associated to a task

    title : string
        a short description associated to a task

    duration : float
        how long in hours it takes to complete a task

    individual : string ("Yes", "No")
        Whether a task concerns the user individually or whether it is a
        general task

    category : string ("Chores", "Non Minerva Academics", "Minerva Academics")
        What type of task a given task is

    dependent : list
        a list of IDs a given task depends on, or in other words which
        tasks must be completed before this task can be completed

    location_based: string ("Yes", "No")
        Whether this task is engaging with Taipei City

    status: string
        shows whether task has been completed or not

    time_constraint_left: float
        the time which a given task must be completed after

    time_constraint_right: float
        the time which a given task must be completed before

    priority: float
        the priority of a given task
    """

    def __init__(
        self,
        id,
        title,
        duration,
        individual,
        category,
```

```

dependent,
location_based,
status=None,
time_constraint_left=None,
time_constraint_right=None,
):
    """
    Parameters
    -----
    As described above (without priority)
    """
    self.id = id
    self.title = title
    self.duration = duration
    self.individual = individual
    self.category = category
    self.dependent = dependent
    self.location_based = location_based
    self.status = status
    self.time_constraint_left = (
        time_constraint_left
    )
    self.time_constraint_right = (
        time_constraint_right
    )
    self.priority = self.calculate_priority()

def calculate_priority(self):
    """
    Takes a given Task object and calculates its utility based on its

    Parameters
    -----
    None

    Returns
    -----
    utility: float
        the expected number of utils user gets from completing the ta.
    """
    priority = 0

    # adding priority values based on the category of a task
    if self.category == "Minerva Academics":
        priority += 15
    elif (
        self.category
        == "Non Minerva Academics"
    ):
        priority += 10
    elif self.category == "Food":
        priority += 6
    elif self.category == "Chores":
        priority += 5
    elif self.category == "Recreation":
        priority += 2

    # adding priority values based on the individuality of a task
    if self.individual == "No":
        priority += 15

```

```

        else:
            priority += 5

        # adding priority values based on whether the task is location ba
    if self.location_based == "Yes":
        priority += 3
    else:
        priority += 1

    if self.duration != 0:
        priority = round(
            priority
            / ((self.duration) ** 0.5),
            2,
        ) # dividing the task by the squareroot of the duration of t
    elif self.duration == 0:
        priority = round(priority)
    return priority

def update_priority_with_dependencies(
    self, all_tasks
):
    """
    Takes a given Task object and updates its utility by summing the

    Parameters
    -----
    all_tasks: list
        list of tasks that the task is in

    Returns
    -----
    utility: float
        the expected number of utils user gets from completing the ta
    """
    for task in all_tasks:
        if self.id in task.dependent:
            self.priority += task.priority
            self.priority = round(
                self.priority, 2
            )

```

```

In [15]: class FixedTask(Task):
    """
    A class that stores properties of a daily fixed task it inherits all
    """

    def __init__(
        self,
        id,
        title,
        duration,
        individual,
        category,
        dependent,
        location_based,
        time_stamp,
    ):
        super().__init__(
            id,

```

```

        title,
        duration,
        individual,
        category,
        dependent,
        location_based,
    )
    self.time_stamp = time_stamp

```

3. Task Schedulers

```

In [17]: class TasksScheduler:
    """
    A class has the properties of a Max heap where the parent is always greater than its children.

    Properties
    -----
        heap : arr
            where priority values are stored

        completed_tasks : set
            where tasks that have been completed are stored, it is a set

        task_with_dependencies : list
            stores tasks whose dependencies have not been completed yet

        left_time_tasks : list
            stores tasks who have a left time requirement until the time

        time : float
            the time the user wakes up, or decides to start executing tasks

        goodnight : float
            the time the user wishes to go to sleep or terminate doing tasks

        weekday: string
            the weekday, used for print statement formatting

        username: string
            the user's name, used for print statement formatting

    """

    def __init__(
        self,
        time,
        goodnight=None,
        weekday=None,
        username=None,
    ):
        self.heap = []
        self.completed_tasks = set()
        self.task_with_dependencies = []
        self.left_time_tasks = []
        self.time = time
        self.goodnight = goodnight
        self.weekday = weekday
        self.username = username

```

```

# CITATION
def left(
    self, i
): # method has been entirely copied (including doc string) from CS1
    """
    Takes the index of the parent node
    and returns the index of the left child node

    Parameters
    -----
    i: int
        Index of parent node

    Returns
    -----
    int
        Index of the left child node
    """
    return 2 * i + 1

# CITATION
def right(
    self, i
): # method has been entirely copied (including doc string) from CS1
    """
    Takes the index of the parent node
    and returns the index of the right child node

    Parameters
    -----
    i: int
        Index of parent node

    Returns
    -----
    int
        Index of the right child node
    """
    return 2 * i + 2

# CITATION
def parent(
    self, i
): # method has been entirely copied (including doc string) from CS1
    """
    Takes the index of the child node
    and returns the index of the parent node

    Parameters
    -----
    i: int
        Index of child node

    Returns
    -----
    int
        Index of the parent node
    """
    return (i - 1) // 2

```



```

def heappush(self, task):
    """
    Insert a task into a priority queue

    Parameters
    -----
    task: int
        The task to be inserted

    Returns
    -----
    None
    """
    self.heap.append(task)
    self.sift_up(len(self.heap) - 1)

def heappop(self):
    """
    Pops the root of the heap if all of its dependencies have been met

    Parameters
    -----
    None

    Returns
    -----
    None
    """
    while (
        self.heap
    ): # checks that the heap is not empty
        for (
            task
        ) in (
            self.left_time_tasks
        ): # if there is a task with left time constraint if the con
            if (
                self.time
                >= task.time_constraint_left
            ):
                self.heappush(task)
                self.left_time_tasks.remove(
                    task
                )

        # pops the root out and stores it in max_task and replaces it
        self.swap(0, len(self.heap) - 1)
        max_task = self.heap.pop()
        self.heapify(
            0
        ) # restores the heap property

        if (
            max_task.time_constraint_left
            and self.time
            < max_task.time_constraint_left
        ): # if time constraint has not yet been satisfied it adds t
            self.left_time_tasks.append(
                max_task
            )

```

```

    )
    continue

    if (
        max_task.time_constraint_right
        == None
    ): # sets up right time constraints for all tasks
        if (
            max_task.category
            == "Non Minerva Academics"
            or max_task.category
            == "Minerva Academics"
        ): # if a task is academic then the constraint will be 2
            max_task.time_constraint_right = (
                self.goodnight - 2
            )
        else:
            max_task.time_constraint_right = (
                self.goodnight
            ) # else the constraint is the time user goes to sle

    if (
        max_task.time_constraint_right
        and (
            self.time + max_task.duration
            > max_task.time_constraint_right
        )
    ): # if the time exceeds the right time constraint we skip t.
        continue

    if all(
        dep in self.completed_tasks
        for dep in max_task.dependent
    ): # checks to make sure the dependencies of the task has be
        self.completed_tasks.add(
            max_task.id
        ) # executes the task
        self.time += (
            max_task.duration
        ) # adds the task duration to the completed tasks

    for task in list(
        self.task_with_dependencies
    ): # checks to see if any new tasks can be pushed back i.
        if all(
            dep
            in self.completed_tasks
            for dep in task.dependent
        ):
            self.heappush(task)
            self.task_with_dependencies.remove(
                task
            )

    return max_task

else: # if not dependencies met it adds the task into task_w
    self.task_with_dependencies.append(
        max_task
    )

```

```

def sift_up(self, i):
    """
    Swaps node i with its parent if it has higher priority than the p

    Parameters
    -----
    i: int
        The index of the node

    Returns
    -----
    None
    """
    while (
        i > 0
        and self.heap[self.parent(i)].priority
        < self.heap[i].priority
    ):
        self.swap(i, self.parent(i))
        i = self.parent(i)

# CITATION
def heapify(
    self, i
):
    # method's docstring has been copied from CS110 Session 13 Pre-Cl
    """
    Creates a max heap from the index given

    Parameters
    -----
    i: int
        The index of of the root node of the subtree to be heapify

    Returns
    -----
    None
    """
    largest = i
    left = self.left(i)
    right = self.right(i)

    if (
        left < len(self.heap)
        and self.heap[left].priority
        > self.heap[largest].priority
    ):
        largest = left
    if (
        right < len(self.heap)
        and self.heap[right].priority
        > self.heap[largest].priority
    ):
        largest = right
    if largest != i:
        self.swap(i, largest)
        self.heapify(largest)

def swap(self, i, j):
    """

```

```

        swaps two nodes i, j

Parameters
-----
i: int
    index of a node
j: int
    index of another node

Returns
-----
None
"""
self.heap[i], self.heap[j] = (
    self.heap[j],
    self.heap[i],
)

```

```

In [18]: # CITATION
# code is based on previous Class (including the cited methods), and doc
class FixedTasksScheduler:
    """
    A class has the properties of a Min heap where the parent is always s

    Properties
    -----
        heap : arr
                where priority values are stored
    """

    def __init__(self):
        self.heap = []

    def left(self, i):
        """
        Takes the index of the parent node
        and returns the index of the left child node

        Parameters
        -----
        i: int
            Index of parent node

        Returns
        -----
        int
            Index of the left child node
        """
        return 2 * i + 1

    def right(self, i):
        """
        Takes the index of the parent node
        and returns the index of the right child node

        Parameters
        -----
        i: int
            Index of parent node

```

```

Returns
-----
int
    Index of the right child node
"""
return 2 * i + 2

def parent(self, i):
    """
    Takes the index of the child node
    and returns the index of the parent node

    Parameters
    -----
    i: int
        Index of child node

    Returns
    -----
    int
        Index of the parent node
    """
    return (i - 1) // 2

def heappush(self, task):
    """
    Insert a task into a priority queue

    Parameters
    -----
    task: int
        The task to be inserted

    Returns
    -----
    None
    """
    self.heap.append(task)
    self.sift_up(len(self.heap) - 1)

def heappop(self):
    """
    Pops out the root of the heap (the task with the earliest time)

    Parameters
    None

    Returns
    -----
    min_task: object
        task with the earliest time
    """
    if not self.heap:
        return None

    self.swap(0, len(self.heap) - 1)
    min_task = self.heap.pop()
    self.heapify(0)
    return min_task

```

```

def sift_up(self, i):
    """
    Swaps node i with its parent if it has a smaller time than the pa

    Parameters
    -----
    i: int
        The index of the node

    Returns
    -----
    None
    """
    while (
        i > 0
        and self.heap[
            self.parent(i)
        ].time_stamp
        > self.heap[i].time_stamp
    ):
        self.swap(i, self.parent(i))
        i = self.parent(i)

def heapify(self, i):
    """
    Creates a min heap from the index given

    Parameters
    -----
    i: int
        The index of of the root node of the subtree to be heapify

    Returns
    -----
    None
    """
    smallest = i
    left = self.left(i)
    right = self.right(i)

    if (
        left < len(self.heap)
        and self.heap[left].time_stamp
        < self.heap[smallest].time_stamp
    ):
        smallest = left
    if (
        right < len(self.heap)
        and self.heap[right].time_stamp
        < self.heap[smallest].time_stamp
    ):
        smallest = right
    if smallest != i:
        self.swap(i, smallest)
        self.heapify(smallest)

def swap(self, i, j):
    """
    swaps two nodes i, j

```

```

Parameters
-----
i: int
    index of a node
j: int
    index of another node

Returns
-----
None
"""
self.heap[i], self.heap[j] = (
    self.heap[j],
    self.heap[i],
)

```

4. Schedule Executor

```

In [20]: class SchedulerExecutor:
    """
    Class creates a schedule based on the max heap of task priority utilization

    Properties
    -----
        scheduler : max heap
            this has the regular tasks in a max heap by priority values

        fixed_tasks_scheduler : max heap
            this has the fixed tasks in a min heap by fixed time

        total_time : float
            stores total time to execute tasks
    """

    def __init__(
        self, scheduler, fixed_tasks_scheduler
    ):
        self.scheduler = scheduler
        self.fixed_tasks_scheduler = (
            fixed_tasks_scheduler
        )
        self.total_time = 0

    def schedule_tasks(self):
        """
        creates the print statement for the schedule by executing either :

        Parameters
        -----
        None

        Returns
        -----
        None
        """
        if (
            self.scheduler.goodnight == None
        ): # if there is no bed time specified sets the end time for tas.

```

```

        self.scheduler.goodnight = 100

    if self.scheduler.weekday == None:
        self.scheduler.weekday = "Daily"

    # produces a nice header for the scheduler
    if self.scheduler.username == None:
        self.scheduler.username = "My"
        print(
            f"[bold]{self.scheduler.username} {self.scheduler.weekday}
        )
    else:
        print(
            f"[bold]{self.scheduler.username}'s {self.scheduler.weekd
        )

    print()

    print(
        f"[bold magenta]Wake Up[/bold magenta] at [bold blue]{OClock(se
    ) # prints when waking up
    while (
        self.scheduler.heap
        or self.fixed_tasks_scheduler.heap
    ): # while there are still tasks
        next_fixed_task = (
            self.get_next_fixed_task()
        ) # gets the next fixed task

        if self.scheduler.heap and (
            not next_fixed_task
            or self.scheduler.time
            < next_fixed_task.time_stamp
        ): # if the time is not yet the next fixed time stamp execut
            self.execute_regular_task(
                next_fixed_task
            )
        elif (
            next_fixed_task
        ): # else execute the fixed task
            self.execute_fixed_task()

    print(
        f"[bold magenta]Sleep[/bold magenta] at [bold blue]{OClock(se
    ) # print time when go to sleep

    if self.scheduler.username != "My":
        print()
        print(
            f"[italic]Well done with your tasks today {self.scheduler
        ) # prints how much time it took to do tasks

def get_next_fixed_task(self):
    """
    gets the next fixed task (the one with the earliest fixed time)

    Parameters
    -----
    None

```



```

Returns
-----
the root of the fixed task heap
"""
return (
    self.fixed_tasks_scheduler.heap[0]
    if self.fixed_tasks_scheduler.heap
    else None
)

def execute_regular_task(
    self, next_fixed_task
):
    """
    executes a regular task if the current time plus the next task wi

Parameters
-----
next_fixed_task: object
    the next task that has a fixed time

Returns
-----
None
"""
task = self.scheduler.heappop()
if task: # if there is still task in heap
    if (
        next_fixed_task
        and self.scheduler.time
        + task.duration
        > next_fixed_task.time_stamp
    ): # if the time it takes to execute the task exceeds the fi
        remaining_duration = (
            task.duration
            - (
                next_fixed_task.time_stamp
                - self.scheduler.time
            )
        )
    print(
        f" □ [bold magenta]{task.title}[/bold magenta] at [bo
    )
    self.scheduler.time = (
        next_fixed_task.time_stamp
    )
    print(
        f" ➡ Interrupt my [bold magenta]{task.title}[/bold m
    )
    if (
        self.execute_fixed_task()
    ): # if the fixed task executed then it resumes the regu
        print(
            f" ➡ Resume [bold magenta]{task.title}[/bold ma
        )
        self.scheduler.time += (
            remaining_duration
        )
        self.scheduler.time -= (
            task.duration

```

```

        )
        self.total_time += task.duration
    else:
        print(
            f" □ [bold magenta]{task.title}[/bold magenta] at [bo
        )
        self.total_time += task.duration

def execute_fixed_task(self):
    """
    executes a fixed task by setting the time to to the tasks time_st

    Parameters
    -----
    None

    Returns
    -----
    None
    """
    fixed_task = (
        self.fixed_tasks_scheduler.heappop()
    )

    if all(
        dep in self.scheduler.completed_tasks
        for dep in fixed_task.dependent
    ):
        print(
            f" □ [bold magenta]{fixed_task.title}[/bold magenta] at [
        )
        self.scheduler.time = (
            fixed_task.time_stamp
            + fixed_task.duration
        )
        self.scheduler.completed_tasks.add(
            fixed_task.id
        )
        self.total_time += fixed_task.duration
        return True
    else:
        print(
            f"Skip {fixed_task.title} because you have not done your
        )
        return False

```

5. Test Cases

Below I have written 5 test cases (1, 2, 2b, 2c, and 3). Test Case 1 makes sure that the Utility of each task is calculated correctly. like I assigned the utility of academics higher than recreation and food, and so naturally this task should be prioritized over the other. The output reflects that. Task 2 checks dependencies, to make sure dependencies are completed priorly. In the test case 2b I update priorities by adding the dependent tasks' priority utility values to the tasks that they depend on (i.e. if task id=1 depends on task id=2, then we add task id=1's priority utility score to task 2). However in the original test case 2, I show how my code novelly sorts out

dependencies without resorting to updating dependancies producing a more efficient code. As I would rather eat Lunch than buy pop then watch a movie, as it has a higher utility than each of these individually, and in test case 2b I skip lunch (so not health!). In test case 2c I rearrange the tasks and change their ID numbers to prove rearranging tasks does not affect the performance of my algorithm. In test case 3 I show how all of my time constraints work. So even though Lunch is prioritized over watching movie and getting popcorn, I cannot get Lunch as it is past 2 pm. And I cannot do anything academically 2 hours before sleeping so even though my Pre-Class work is prioritized I do not do it.

```
In [23]: # TEST CASE 1

tasks = [
    Task(
        id=1,
        title="Watch Movie",
        duration=1,
        individual="Yes",
        category="Recreation",
        dependent=[],
        location_based="No",
    ),
    Task(
        id=2,
        title="CS110 Pre-Class Work",
        duration=1,
        individual="Yes",
        category="Minerva Academics",
        dependent=[],
        location_based="No",
    ),
    Task(
        id=3,
        title="Lunch",
        duration=1,
        individual="Yes",
        category="Food",
        dependent=[],
        location_based="No",
        time_constraint_right=14,
    ),
]

fixed_tasks = []

# creates instance objects of task and fixed tasks class heaps
test_case1_tasks = TasksScheduler(
    time=11, goodnight=22
)
test_case1_fixed_tasks = FixedTasksScheduler()

for (
    task
) in (
    tasks
): # pushes all the tasks in the scheduler heap
    test_case1_tasks.heappush(task)
```

```

for (
    fixed_task
) in (
    fixed_tasks
): # pushes all the tasks in the fixed scheduler heap
    test_case1_fixed_tasks.heappush(fixed_task)

# sets up an instance of the class SchedulerExecutor
test1_schedule = SchedulerExecutor(
    test_case1_tasks, test_case1_fixed_tasks
)
test1_schedule.schedule_tasks()

```

My Daily Schedule 🐛

Wake Up at 11:00 AM

- CS110 Pre-Class Work at 11:00 AM. Priority utility of 21.0
- Lunch at 12:00 PM. Priority utility of 12.0
- Watch Movie at 1:00 PM. Priority utility of 8.0

Sleep at 10:00 PM

```

In [24]: # TEST CASE 2

tasks = [
    Task(
        id=1,
        title="Watch Movie",
        duration=1,
        individual="Yes",
        category="Recreation",
        dependent=[2],
        location_based="No",
    ),
    Task(
        id=2,
        title="Buy Popcorn",
        duration=1,
        individual="Yes",
        category="Recreation",
        dependent=[],
        location_based="No",
    ),
    Task(
        id=3,
        title="CS110 Pre-Class Work",
        duration=1,
        individual="Yes",
        category="Minerva Academics",
        dependent=[],
        location_based="No",
    ),
    Task(
        id=4,
        title="Lunch",
        duration=1,
        individual="Yes",
        category="Food",
        dependent=[],
    ),
]

```

```

        location_based="No",
        time_constraint_right=14,
    ),
]

fixed_tasks = []

# creates instance objects of task and fixed tasks class heaps
test_case2_tasks = TasksScheduler(
    time=11.5, goodnight=22
)
test_case2_fixed_tasks = FixedTasksScheduler()

for (
    task
) in (
    tasks
): # pushes all the tasks in the scheduler heap
    test_case2_tasks.heappush(task)

for (
    fixed_task
) in (
    fixed_tasks
): # pushes all the tasks in the fixed scheduler heap
    test_case2_fixed_tasks.heappush(fixed_task)

# sets up an instance of the class SchedulerExecutor
test2_schedule = SchedulerExecutor(
    test_case2_tasks, test_case2_fixed_tasks
)
test2_schedule.schedule_tasks()

```

My Daily Schedule 🐛

Wake Up at 11:30 AM

- ❑ CS110 Pre-Class Work at 11:30 AM. Priority utility of 21.0
- ❑ Lunch at 12:30 PM. Priority utility of 12.0
- ❑ Buy Popcorn at 1:30 PM. Priority utility of 8.0
- ❑ Watch Movie at 2:30 PM. Priority utility of 8.0

Sleep at 10:00 PM

```

In [25]: # TEST CASE 2B

tasks = [
    Task(
        id=1,
        title="Watch Movie",
        duration=1,
        individual="Yes",
        category="Recreation",
        dependent=[2],
        location_based="No",
    ),
    Task(
        id=2,
        title="Buy Popcorn",
        duration=1,

```

```

        individual="Yes",
        category="Recreation",
        dependent=[],
        location_based="No",
    ),
    Task(
        id=3,
        title="CS110 Pre-Class Work",
        duration=1,
        individual="Yes",
        category="Minerva Academics",
        dependent=[],
        location_based="No",
    ),
    Task(
        id=4,
        title="Lunch",
        duration=1,
        individual="Yes",
        category="Food",
        dependent=[],
        location_based="No",
        time_constraint_right=14,
    ),
]

fixed_tasks = []

# creates instance objects of task and fixed tasks class heaps
test_case2b_tasks = TasksScheduler(
    time=11.5, goodnight=22
)
test_case2b_fixed_tasks = FixedTasksScheduler()

for (
    task
) in tasks: # updates priorities for all tasks
    task.update_priority_with_dependencies(tasks)

for (
    fixed_task
) in (
    fixed_tasks
): # updates priorities for all fixed tasks
    fixed_task.update_priority_with_dependencies(
        fixed_tasks
    )

for (
    task
) in (
    tasks
): # pushes all the tasks in the scheduler heap
    test_case2b_tasks.heappush(task)

for (
    fixed_task
) in (
    fixed_tasks
): # pushes all the tasks in the fixed scheduler heap

```

```

test_case2b_fixed_tasks.heappush(fixed_task)

# sets up an instance of the class SchedulerExecutor
test2b_schedule = SchedulerExecutor(
    test_case2b_tasks, test_case2b_fixed_tasks
)
test2b_schedule.schedule_tasks()

```

My Daily Schedule 🐛

Wake Up at **11:30 AM**

- **CS110 Pre-Class Work** at **11:30 AM**. Priority utility of **21.0**
- **Buy Popcorn** at **12:30 PM**. Priority utility of **16.0**
- **Watch Movie** at **1:30 PM**. Priority utility of **8.0**

Sleep at **10:00 PM**

```

In [26]: # TEST CASE 2C

tasks = [
    Task(
        id=1,
        title="CS110 Pre-Class Work",
        duration=1,
        individual="Yes",
        category="Minerva Academics",
        dependent=[],
        location_based="No",
    ),
    Task(
        id=2,
        title="Watch Movie",
        duration=1,
        individual="Yes",
        category="Recreation",
        dependent=[4],
        location_based="No",
    ),
    Task(
        id=3,
        title="Lunch",
        duration=1,
        individual="Yes",
        category="Food",
        dependent=[],
        location_based="No",
        time_constraint_right=14,
    ),
    Task(
        id=4,
        title="Buy Popcorn",
        duration=1,
        individual="Yes",
        category="Recreation",
        dependent=[],
        location_based="No",
    ),
]

fixed_tasks = []

```

```

# creates instance objects of task and fixed tasks class heaps
test_case2c_tasks = TasksScheduler(
    time=11.5, goodnight=22
)
test_case2c_fixed_tasks = FixedTasksScheduler()

for (
    task
) in (
    tasks
): # pushes all the tasks in the scheduler heap
    test_case2c_tasks.heappush(task)

for (
    fixed_task
) in (
    fixed_tasks
): # pushes all the tasks in the fixed scheduler heap
    test_case2c_fixed_tasks.heappush(fixed_task)

# sets up an instance of the class SchedulerExecutor
test2c_schedule = SchedulerExecutor(
    test_case2c_tasks, test_case2c_fixed_tasks
)
test2c_schedule.schedule_tasks()

```

My Daily Schedule 🐜

Wake Up at 11:30 AM

- **CS110 Pre-Class Work** at **11:30 AM**. Priority utility of **21.0**
- **Lunch** at **12:30 PM**. Priority utility of **12.0**
- **Buy Popcorn** at **1:30 PM**. Priority utility of **8.0**
- **Watch Movie** at **2:30 PM**. Priority utility of **8.0**

Sleep at 10:00 PM

```

In [27]: # TEST CASE 3

tasks = [
    Task(
        id=1,
        title="Watch Movie",
        duration=1,
        individual="Yes",
        category="Recreation",
        dependent=[2],
        location_based="No",
    ),
    Task(
        id=2,
        title="Buy Popcorn",
        duration=1,
        individual="Yes",
        category="Recreation",
        dependent=[],
        location_based="No",
    ),
    Task(

```



```

        id=4,
        title="Lunch",
        duration=1,
        individual="Yes",
        category="Food",
        dependent=[],
        location_based="No",
        time_constraint_right=14,
    ),
    Task(
        id=3,
        title="CS110 Pre-Class Work",
        duration=1,
        individual="Yes",
        category="Minerva Academics",
        dependent=[],
        location_based="No",
    ),
]

fixed_tasks = []

# creates instance objects of task and fixed tasks class heaps
test_case3_tasks = TasksScheduler(
    time=18, goodnight=20
)
test_case3_fixed_tasks = FixedTasksScheduler()

for (
    task
) in (
    tasks
): # pushes all the tasks in the scheduler heap
    test_case3_tasks.heappush(task)

for (
    fixed_task
) in (
    fixed_tasks
): # pushes all the tasks in the fixed scheduler heap
    test_case3_fixed_tasks.heappush(fixed_task)

# sets up an instance of the class SchedulerExecutor
test3_schedule = SchedulerExecutor(
    test_case3_tasks, test_case3_fixed_tasks
)
test3_schedule.schedule_tasks()

```

My Daily Schedule 🐛

Wake Up at **6:00 PM**

- ☐ **Buy Popcorn** at **6:00 PM**. Priority utility of **8.0**
- ☐ **Watch Movie** at **7:00 PM**. Priority utility of **8.0**

Sleep at **8:00 PM**

6. Mara's Monday Schedule (LBA Portion)

How I engage with the city in these LBA Tasks

Biking to NTU: Biking through the city as I reach my final destination engages me with the city by exposing me to street life culture in Taiwan, as without biking for example I would not notice how Taiwan is motorbike dominated on streets.

Night Market: Nights markets provide a way to experience night life in Taiwan, they not only provide a window to try out all kinds of taiwan food, but observe different products like clothing or accessories, as well as see people just having fun like dancing as a community

Taiwanese Pickle Sandwich: The Taiwanese Pickle sandwich is one of a kind it engages me with the city by trying out local cuisine

Lunch: Similar as the sandwich Lunch engages me with the city as usually I just find lunch close to wherever I am, whatever inspires me. So it engages me with the given location by seeing how food differs accross taiwan. Like around NTu for instance there is cheap and western food to accomodate young broke college students, and around financial shopping district shtere are very expensive restaurants to accodmate consumers.

```
In [30]: tasks = [
    Task(
        id=1,
        title="Get Dressed",
        duration=1,
        individual="Yes",
        category="Chores",
        dependent=[],
        location_based="No",
    ),
    Task(
        id=2,
        title="NTU Library Study Session",
        duration=1,
        individual="No",
        category="Non Minerva Academics",
        dependent=[1, 4, 9],
        location_based="No",
    ),
    Task(
        id=3,
        title="NTU Piano",
        duration=0.5,
        individual="No",
        category="Recreation",
        dependent=[1, 4],
        location_based="No",
    ),
    Task(
        id=4,
        title="Bike to NTU",
        duration=0.5,
        individual="Yes",
```

```

        category="Chores",
        dependent=[1],
        location_based="Yes",
    ),
    Task(
        id=5,
        title="Read",
        duration=0.75,
        individual="Yes",
        category="Recreation",
        dependent=[1],
        location_based="No",
    ),
    Task(
        id=6,
        title="Lunch",
        duration=1.5,
        individual="Yes",
        category="Food",
        dependent=[1, 10, 12],
        location_based="Yes",
        time_constraint_right=15,
    ),
    Task(
        id=7,
        title="AI Lab Study",
        duration=1,
        individual="No",
        category="Non Minerva Academics",
        dependent=[1, 9],
        location_based="No",
    ),
    Task(
        id=8,
        title="Night Market",
        duration=2,
        individual="No",
        category="Recreation",
        dependent=[1, 4, 10],
        location_based="Yes",
        time_constraint_left=16,
    ),
    Task(
        id=9,
        title="Charge Electronics",
        duration=0.75,
        individual="Yes",
        category="Chores",
        dependent=[1],
        location_based="No",
    ),
    Task(
        id=10,
        title="Extract Cash",
        duration=0.1,
        individual="Yes",
        category="Chores",
        dependent=[1],
        location_based="No",
    ),

```

```
Task(  
    id=11,  
    title="Breakfast",  
    duration=1,  
    individual="Yes",  
    category="Food",  
    dependent=[1, 10, 12],  
    location_based="No",  
    time_constraint_right=10,  
) ,  
Task(  
    id=12,  
    title="Brush Teeth",  
    duration=0.05,  
    individual="Yes",  
    category="Chores",  
    dependent=[],  
    location_based="No",  
) ,  
Task(  
    id=13,  
    title="CS110 Pre-Class Work",  
    duration=3,  
    individual="Yes",  
    category="Minerva Academics",  
    dependent=[9],  
    location_based="No",  
) ,  
Task(  
    id=14,  
    title="CS111 Pre-Class Work",  
    duration=1,  
    individual="Yes",  
    category="Minerva Academics",  
    dependent=[9],  
    location_based="No",  
) ,  
Task(  
    id=15,  
    title="SS111 Pre-Class Work",  
    duration=0.5,  
    individual="Yes",  
    category="Minerva Academics",  
    dependent=[9],  
    location_based="No",  
) ,  
Task(  
    id=16,  
    title="Taiwanese Pickle Sandwich",  
    duration=0.25,  
    individual="Yes",  
    category="Food",  
    dependent=[1, 4, 8, 10],  
    location_based="Yes",  
) ,  
Task(  
    id=17,  
    title="Watch a Movie",  
    duration=2,  
    individual="Yes",
```

```

        category="Recreation",
        dependent=[],
        location_based="No",
        time_constraint_left=20,
    ),
]

fixed_tasks = [
    FixedTask(
        id=18,
        title="AI Lab Meeting",
        duration=1,
        individual="No",
        category="Non Minerva Academics",
        dependent=[7],
        location_based="No",
        time_stamp=18,
    )
]

```

```

In [31]: # creates instance objects of task and fixed tasks class heaps
maras_monday_tasks = TasksScheduler(
    time=10,
    goodnight=24,
    weekday="Monday",
    username="Mara",
)
maras_fixed_tasks = FixedTasksScheduler()

# I personally would decide to ignore updating priorities as my code work.

for (
    task
) in tasks: # updates priorities for all tasks
    task.update_priority_with_dependencies(tasks)

for (
    fixed_task
) in (
    fixed_tasks
): # updates priorities for all fixed tasks
    fixed_task.update_priority_with_dependencies(
        fixed_tasks
    )

for (
    task
) in (
    tasks
): # pushes all the tasks in the scheduler heap
    maras_monday_tasks.heappush(task)

for (
    fixed_task
) in (
    fixed_tasks
): # pushes all the tasks in the fixed scheduler heap
    maras_fixed_tasks.heappush(fixed_task)

```

```
In [32]: # sets up an instance of the class SchedulerExecutor
my_schedule = SchedulerExecutor(
    maras_monday_tasks, maras_fixed_tasks
)
my_schedule.schedule_tasks()
```

Mara's Monday Schedule 🐜

Wake Up at 10:00 AM

- **Get Dressed** at 10:00 AM. Priority utility of 229.14
- **Extract Cash** at 11:00 AM. Priority utility of 128.36
- **Charge Electronics** at 11:06 AM. Priority utility of 127.52
- **Bike to NTU** at 11:51 AM. Priority utility of 111.98
- **Brush Teeth** at 12:21 PM. Priority utility of 72.62
- **SS111 Pre-Class Work** at 12:24 PM. Priority utility of 29.7
- **NTU Library Study Session** at 12:54 PM. Priority utility of 26.0
- **AI Lab Study** at 1:54 PM. Priority utility of 26.0
- **NTU Piano** at 2:54 PM. Priority utility of 25.46
- **CS111 Pre-Class Work** at 3:24 PM. Priority utility of 21.0
- **Night Market** at 4:24 PM, Priority utility of 42.14
 - ➡ Interupt my **Night Market** for my **AI Lab Meeting** at 6:00 PM
- **AI Lab Meeting** at 6:00 PM. Priority utility of 26.0
 - ➡ Resume **Night Market** at 7:00 PM
- **Taiwanese Pickle Sandwich** at 7:24 PM. Priority utility of 28.0
- **Read** at 7:39 PM. Priority utility of 9.24
- **Watch a Movie** at 8:24 PM. Priority utility of 5.66

Sleep at 0:00 AM

Well done with your tasks today Mara, you spent a total of 12.4 hour
See you again tomorrow 😊

7 Time Complexity Check

```
In [36]: import random
import time
import matplotlib.pyplot as plt

def generator1(n):
    tasks = []
    for i in range(n):
        task_id = i + 1
        duration = random.uniform(0.5, 2.0)
        tasks.append(
            Task(
                id=task_id,
                title=f"Task {task_id}",
                duration=duration,
                individual="Yes",
                category="Recreation",
                dependent=[],
                location_based="No",
```

```

        status="Not started",
        time_constraint_left=None,
        time_constraint_right=None,
    )
)
return tasks

def time_efficiency1(n):
    tasks = generator1(n)

    task_scheduler = TasksScheduler(time=9, goodnight=22)

    start_time = time.time()
    for task in tasks:
        task_scheduler.heappush(task)
    end_time = time.time()

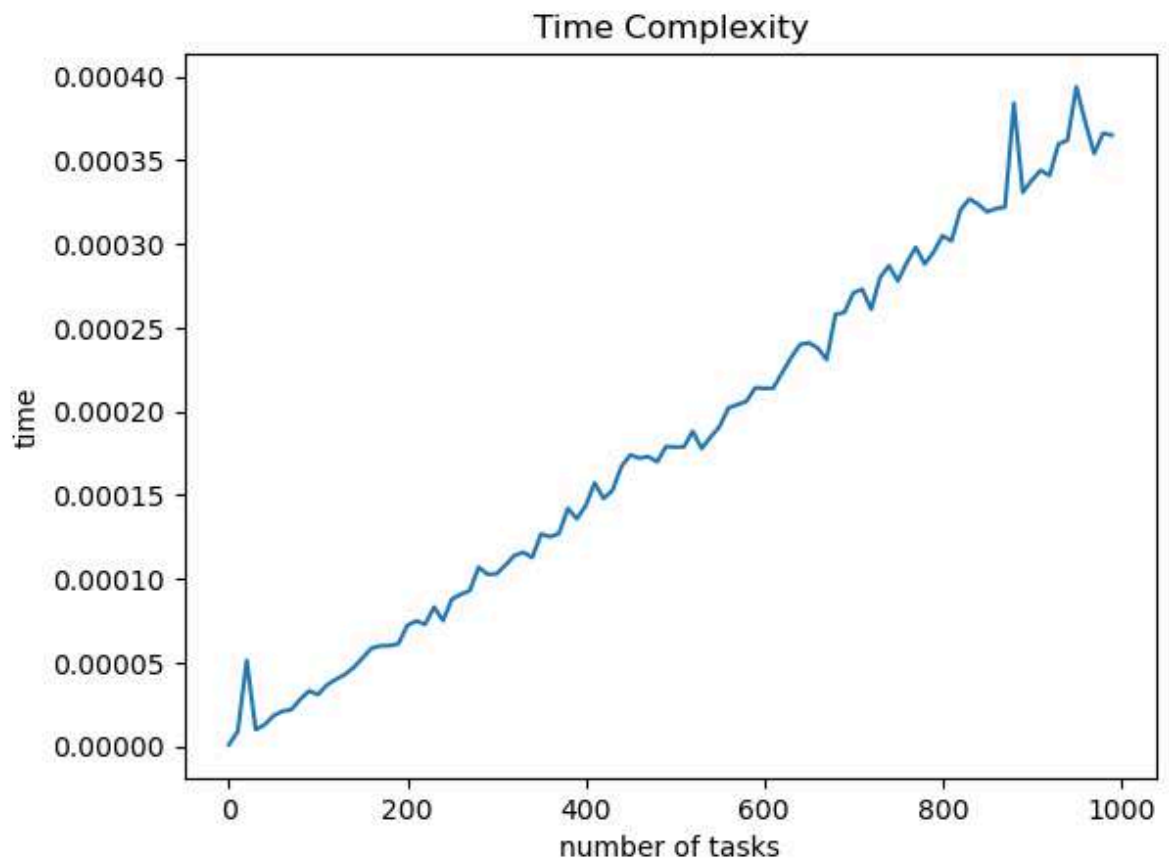
    return end_time - start_time

sizes = []
for i in range(100):
    sizes.append(i * 10)

times = [time_efficiency1(size) for size in sizes]

plt.plot(sizes, times)
plt.xlabel("number of tasks")
plt.ylabel("time")
plt.title("Time Complexity")
plt.show()

```



```

In [68]: def generator2(n):
    tasks = []
    for i in range(n):
        task_id = i + 1
        duration = random.uniform(0.5, 2.0)
        if task_id == 1:
            tasks.append(
                Task(
                    id=task_id,
                    title=f"Task {task_id}",
                    duration=duration,
                    individual="Yes",
                    category="Recreation",
                    dependent=[],
                    location_based="No",
                    status="Not started",
                    time_constraint_left=None,
                    time_constraint_right=None,
                )
            )
        else:
            tasks.append(
                Task(
                    id=task_id,
                    title=f"Task {task_id}",
                    duration=duration,
                    individual="Yes",
                    category="Recreation",
                    dependent=[task_id - 1],
                    location_based="No",
                    status="Not started",
                    time_constraint_left=None,
                    time_constraint_right=None,
                )
            )

    return tasks

def time_efficiency2(n):
    tasks = generator2(n)

    task_scheduler = TasksScheduler(time=9, goodnight=22)

    start_time = time.time()
    for task in tasks:
        task_scheduler.heappush(task)
    end_time = time.time()

    return end_time - start_time

sizes = []
for i in range(100):
    sizes.append(i * 10)

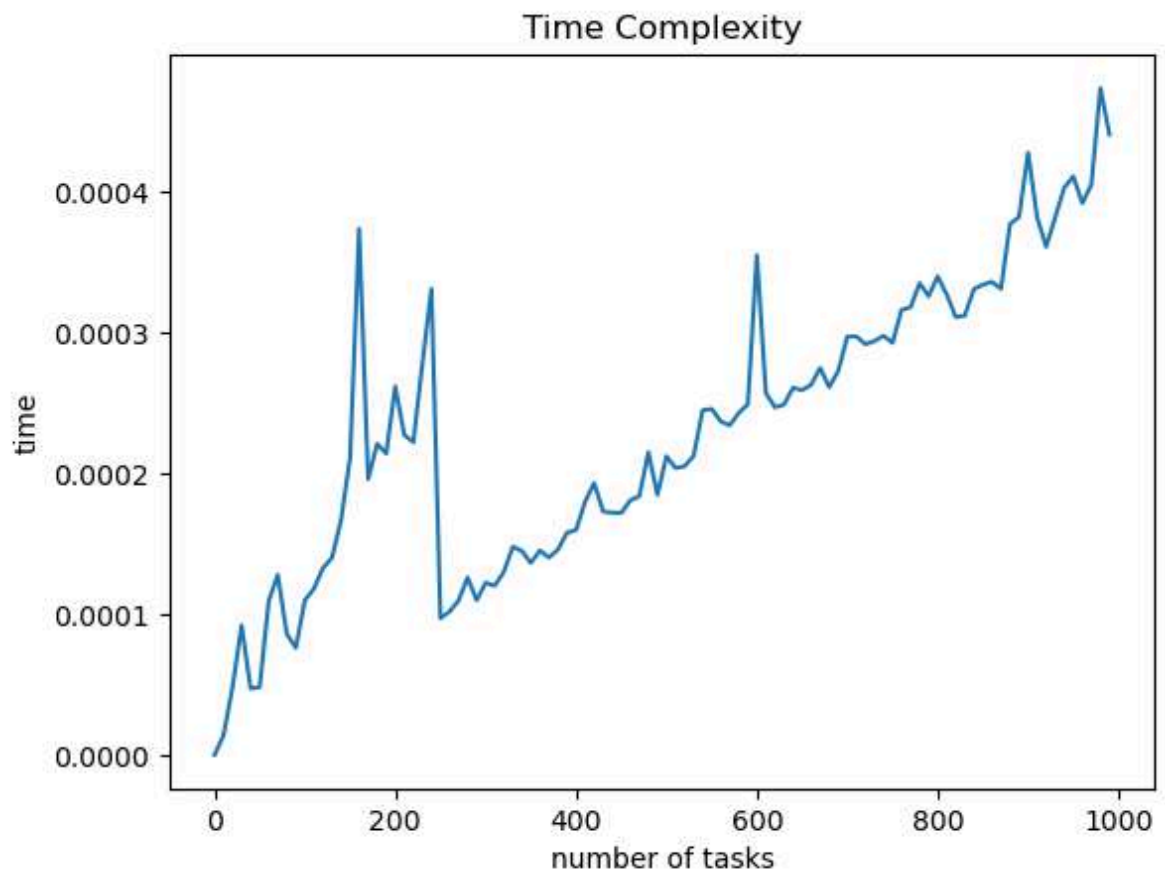
times = [time_efficiency2(size) for size in sizes]

plt.plot(sizes, times)

```



```
plt.xlabel("number of tasks")
plt.ylabel("time")
plt.title("Time Complexity")
plt.show()
```



```
In [74]: def generator3(n):
    tasks = []
    for i in range(n):
        task_id = i + 1
        duration = random.uniform(0.5, 2.0)
        if task_id == 1:
            tasks.append(
                Task(
                    id=task_id,
                    title=f"Task {task_id}",
                    duration=duration,
                    individual="Yes",
                    category="Recreation",
                    dependent=[],
                    location_based="No",
                    status="Not started",
                    time_constraint_left=None,
                    time_constraint_right=None,
                )
            )
        else:
            tasks.append(
                Task(
                    id=task_id,
                    title=f"Task {task_id}",
                    duration=duration,
                    individual="Yes",
                    category="Recreation",
```

```

        dependent=[random.randint(1, 100)],
        location_based="No",
        status="Not started",
        time_constraint_left=None,
        time_constraint_right=None,
    )
)

return tasks

def time_efficiency3(n):
    tasks = generator3(n)

    task_scheduler = TasksScheduler(time=9, goodnight=22)

    start_time = time.time()
    for task in tasks:
        task_scheduler.heappush(task)
    end_time = time.time()

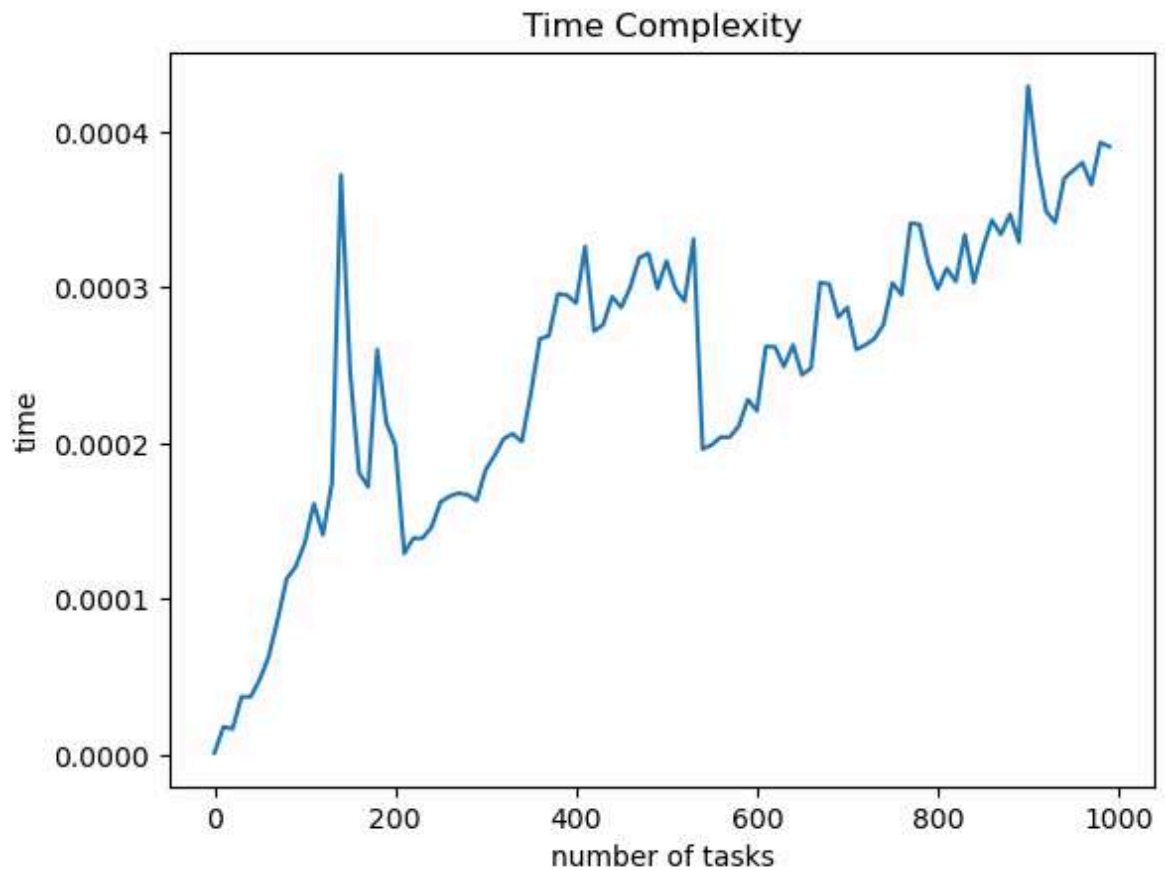
    return end_time - start_time

sizes = []
for i in range(100):
    sizes.append(i * 10)

times = [time_efficiency3(size) for size in sizes]

plt.plot(sizes, times)
plt.xlabel("number of tasks")
plt.ylabel("time")
plt.title("Time Complexity")
plt.show()

```



```
In [86]: def generator4(n):
    tasks = []
    for i in range(n):
        task_id = i + 1
        duration = random.uniform(0.5, 2.0)
        if task_id == n:
            tasks.append(
                Task(
                    id=task_id,
                    title=f"Task {task_id}",
                    duration=duration,
                    individual="Yes",
                    category="Recreation",
                    dependent=[],
                    location_based="No",
                    status="Not started",
                    time_constraint_left=None,
                    time_constraint_right=None,
                )
            )
        else:
            tasks.append(
                Task(
                    id=task_id,
                    title=f"Task {task_id}",
                    duration=duration,
                    individual="Yes",
                    category="Recreation",
                    dependent=[n - task_id + 1],
                    location_based="No",
                    status="Not started",
                    time_constraint_left=None,
```

```

        time_constraint_right=None,
    )
)

return tasks

def time_efficiency4(n):
    tasks = generator4(n)

    task_scheduler = TasksScheduler(time=9, goodnight=22)

    start_time = time.time()
    for task in tasks:
        task_scheduler.heappush(task)
    end_time = time.time()

    return end_time - start_time

sizes = []
for i in range(100):
    sizes.append(i * 10)

times = [time_efficiency4(size) for size in sizes]

plt.plot(sizes, times)
plt.xlabel("number of tasks")
plt.ylabel("time")
plt.title("Time Complexity")
plt.show()

```

